3.2 Protected visibility and abstract classes

Chapter 19 adds `protected` to the visibility choices from Class 2. We can share an implementation detail with subclasses without making it part of the public API.

Compare the access rules

Visibility	Declaring class	Subclass methods	Unrelated calling code	UML symbol
`public`	Yes	Yes	Yes	`+`
`protected`	Yes	Yes	No	`#`
`private`	Yes	No direct access	No	`-`

Private parent state can still exist in a child object. Parent methods can read and change that state; child methods cannot access it directly. Changing a property to protected gives every descendant more power, including the power to bypass a setter's validation. Keep state private when subclasses can work through methods instead.

Demonstrate protected access

Run this complete example in a fresh `3-2-visibility.php` file:

```php
<?php
declare(strict_types=1);

class Vehicle
{
    public function __construct(
        protected string $makeModel,
        private int $numPassengers,
    ) {
    }

    public function getNumPassengers(): int
    {
        return $this->numPassengers;
    }
}

class Car extends Vehicle
{
    public function getLabel(): string
    {
        return $this->makeModel . ' seats ' . $this->getNumPassengers();
    }
}
```

```php
$car = new Car('City Compact', 4);
echo $car->getLabel(), PHP_EOL;
```

Output: `City Compact seats 4`.

Trace: Car inherits the constructor. That constructor initializes the protected and private properties. Car's `getLabel()` reads the protected name directly and uses the inherited public getter for the private passenger count.

Try these changes one at a time, then restore the working code:

1. Replace `$this->getNumPassengers()` with `$this->numPassengers`. PHP reports an undefined property warning because that private parent property is not directly accessible in Car's scope.
2. Add `echo $car->makeModel;` to the entry code. PHP throws an Error for accessing a protected property from outside the hierarchy.
3. Restore both changes. Explain that making every property public would remove the errors but also remove the intended boundary.

UML diagram

The UML diagram on page 366 uses `#makeModel` for protected state. An annotation such as `«get/set»` is shorthand for accessor methods; it is a diagram convention, not PHP code. It also is not the `#[...]` attribute syntax we will learn in 3.4.

Make the superclass abstract

Change only `class Vehicle` to `abstract class Vehicle`. Run the file again: Car still works. Now temporarily append:

```php
$vehicle = new Vehicle('Generic', 1);
```

PHP throws an Error: an abstract class cannot be instantiated. Remove that line. An abstract class can still contain properties, a constructor, and fully implemented methods. It does not need an abstract method. The keyword expresses that the design requires a more specific type of object.

Brief extension: abstract methods

This supports the chapter's abstract-class idea but goes beyond its main examples. Add this declaration inside Vehicle:

```php
abstract public function getLabel(): string;
```

It has a semicolon and no body. Car already supplies a compatible implementation, so it remains concrete. A concrete subclass that omits `getLabel()` will fail; it must implement the method or itself remain abstract. There is no parent body to call for an abstract method.